

ASD 2024 - I Appello

Prova di programmazione (12 punti)

1. (2 punti)

Sia dato un tipo ADT di prima classe SLIST (lista ordinata) con item corrispondente alla typedef qui riportata:

```
typedef struct {char name[16]; int val;} Item;
```

L'item consiste quindi in un nome (contenente solo caratteri alfabetici) a cui è associato un valore intero. Gli item in lista sono ordinati in modo crescente in base al campo name.

Scrivere una funzione SLISTmerge, avente il prototipo seguente:

```
SLIST SLISTmerge(SLIST a, SLIST b);
```

I due parametri a e b sono liste ordinate. Si noti che è possibile che una stessa stringa compaia come name in più di un item sia in a che in b (o in entrambe).

La funzione genera una terza lista, i cui item contengono (nei campi name) tutte e sole le stringhe presenti in a e in b. Ma non sono possibili ripetizioni: quando una stringa compare più volte in a e/o in b, la si riporta una volta sola nel risultato, associando come campo val la somma dei valori associati alla stringa in a e/o b.

Si fornisca la definizione del tipo SLIST, del nodo in lista, e si scriva la funzione. Qualora si usi una funzione newNode, non è necessario scriverla.

Esempio:

a) ("roma", 7), ("torino", 4), ("zagabria", 5)

b) ("roma", 3), ("torino", 3), ("torino", 2), ("venezia", 10)

Risultato: ("roma", 10), ("torino", 9), ("venezia", 10), ("zagabria", 5)

2. (4 punti)

Sia dato un BST di interi (ADT di prima classe). Si scriva la funzione che genera un vettore (da allocare dinamicamente, senza riallocazione) di puntatori ai nodi del BST. Il vettore di puntatori deve essere ordinato secondo questo criterio: profondità crescente e, a pari profondità, valori (dell'intero contenuto nei nodi) crescenti.

La funzione deve essere richiamabile come

```
pnodes = BSTlevelizedNodes(b, &n);
```

Dove b è il BST, pnodes il (vettore) risultato e n il numero di nodi.

È richiesta la definizione dei tipo BST e del nodo.

Non è ammesso l'uso di funzioni di libreria.

3. (6 punti)

È dato un elenco di parole senza duplicati (vettore di stringhe).

Le stringhe in elenco possono essere utilizzate per generare stringhe più lunghe mediante concatenazione. Una concatenazione è una stringa generata dalla sequenza ordinata di tutte o parte delle stringhe in elenco prese al più una volta.

Si scriva una funzione bestConcat, che, dato l'elenco, generi la stringa più lunga ottenibile mediante concatenazione, rispettando un vincolo: date due parole consecutive nella sequenza, se la prima termina con vocale, la seconda non può iniziare con vocale. Se la prima termina con consonante, la seconda non può iniziare con consonante. Ai fini della bontà del risultato la lunghezza si misura in

numero di caratteri (quindi non di parole).

Nota bene: le due domande a seguire **NON** sono facoltative. Assicurarsi di rispondere a tutte le richieste.

```
char *bestConcat(char **parole, int nparole) {  
    }  
}
```

Si giustifichi la scelta del modello combinatorio adottato.

Si descrivano i criteri di pruning adottati o il motivo della loro assenza.